MODULE 3 · JAVASCRIPT FUNDAMENTALS

Session 6: JavaScript Basics

Frontend Development Bootcamp 2026 ·

DATE	TIME	PLATFORM	DURATION
Saturday	4:00 PM – 7:00 PM	Zoom (Live)	3 Hours

Prepared & Presented by

Utkarsh Kushwaha

About the Presenter

UK

Utkarsh Kushwaha

Full Stack Developer | Technical Trainer

Experienced in building responsive and interactive web applications using modern technologies and actively involved in technical communities, workshops, and student learning initiatives

TECHNICAL SKILLS

HTML | CSS | JavaScript | React | Node.js | MongoDB | Express | Docker

WHY THIS SESSION MATTERS

HTML gives structure and CSS gives style — but neither can react to a click, validate a form, or make a decision. JavaScript is what brings your pages to life. Today we lay the foundation: variables, data, operators, decisions and repetition.

Learning Objectives

By the End of This Session, Students Will Be Able To:

- ✓ Explain what JavaScript does and how it connects to HTML
- ✓ Declare and use variables correctly with let and const
- ✓ Identify and work with JavaScript's core data types
- ✓ Use arithmetic, comparison and logical operators confidently
- ✓ Write decision-making logic using if / else if / else
- ✓ Repeat actions using for, while and do-while loops

Outcome: Write basic JavaScript programs and implement simple decision-making logic.

Session Agenda

Today's 3-hour block: ~2.5 hrs teaching + live coding, ~30 min doubts & discussion.

JS

Introduction to JavaScript

role in the web, adding JS to a page

x

Variables & Constants

let, const, var, naming rules

#

Data Types

string, number, boolean, undefined, null

+

Operators

arithmetic, comparison, logical, assignment

?

Conditional Statements

if, else if, else, switch

↻

Loops

for, while, do-while

★

Real-World Mini Program

putting it all together

Hands-on Practice + Doubts

build along, ask questions

What is JavaScript?

You've built structure with HTML and style with CSS — now we add behaviour.

A programming language

Unlike HTML/CSS, JS has logic: variables, conditions, loops, functions.

Runs in the browser

No installation needed — every browser has a built-in JS engine.

Makes pages interactive

Clicks, form validation, dynamic content, animations — all JS.

The 'muscles' of the web

HTML = skeleton, CSS = skin, JavaScript = movement and behaviour.

Outcome: JavaScript is what turns a static page into an application that responds to the user.

Adding JavaScript to a Page

Three ways to bring JS into HTML — one is the professional default.

CODE

```
<!-- Inline (avoid) -->
<button onclick="alert('Hi')">Click</button>

<!-- Internal -->
<script>
  console.log("Hello from JS!");
</script>

<!-- External (preferred) -->
<script src="script.js"></script>
```

Inline JS

Directly in an HTML attribute — quick but messy, avoid it

Internal <script>

Inside the HTML file — fine for small demos

External file (.js)

Best practice: separate script.js, linked in HTML

console.log()

Prints output to the browser DevTools console

Outcome: Always link an external .js file — it keeps HTML and logic cleanly separated.

Variables & Constants

Named containers that store data your program can use and change.

CODE

```
let age = 21;
age = 22;              // ✓ can reassign

const name = "Utkarsh";
name = "Other";        // ✗ Error!

var oldWay = true;     // legacy, avoid
```

`let`	Value can change later — use for anything that updates
`const`	Value cannot be reassigned — use by default
`var`	Old syntax from before 2015 — avoid in new code

NAMING RULES

- Must start with a letter, _ or $
- Cannot start with a number
- Case-sensitive: age ≠ Age
- Use camelCase: studentName
- Be descriptive: totalPrice, not x

Outcome: Default to const. Only use let when you know the value will change. Never use var.

Data Types

Every value in JavaScript belongs to one of these core types.

CODE

```
let name = "Utkarsh";      // String
let age = 21;               // Number
let price = 499.99;         // Number
let isStudent = true;       // Boolean
let city;                   // Undefined
let score = null;           // Null

console.log(typeof age);    // "number"
```

String	"text" in quotes
Number	integers & decimals
Boolean	true or false
Undefined	declared, no value yet
Null	intentionally empty

Outcome: Use typeof to check a value's data type — you'll use it constantly while debugging.

Operators — Arithmetic & Assignment

The building blocks of doing math and updating variables.

CODE

```
let a = 10, b = 3;
a + b;    // 13   addition
a - b;    // 7    subtraction
a * b;    // 30   multiplication
a / b;    // 3.33 division
a % b;    // 1    remainder (modulus)
a ** b;   // 1000 exponent

let total = 0;
total += 5;   // same as total = total + 5
```

QUICK REFERENCE

`+ - * /`

add, subtract, multiply, divide

`%`

modulus — remainder after division

`**`

exponent — a to the power of b

`+= -= *= /=`

shorthand: update and reassign

`++  --`

increment / decrement by 1

Outcome: Modulus (%) is the most useful "new" operator — it's key for even/odd checks and cycles.

Operators — Comparison & Logical

These are what power every decision your program will make.

CODE

```
let age = 20;

age === 20;     // true  (strict equal)
age !== 18;     // true  (not equal)
age > 18;       // true
age <= 20;      // true

let isAdult = age >= 18;
let hasID = true;

isAdult && hasID;   // true — both must be true
isAdult || hasID;   // true — at least one true
!isAdult;           // false — flips the value
```

`===` / `!==`	strict equal / not equal (use these, not == / !=)
`>` `<` `>=` `<=`	greater than, less than, and or-equal-to
`&&`	AND — true only if both sides are true
`||`	OR — true if at least one side is true
`!`	NOT — flips true to false and vice versa

Outcome: Always use === and !== in JavaScript — == can silently convert types and cause bugs.

Conditional Statements

This is how a program makes decisions — the heart of programming logic.

CODE

```
let marks = 72;

if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 75) {
  console.log("Grade: B");
} else if (marks >= 60) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}

// Output: Grade: C
```

HOW IT FLOWS

- if — checked first; runs if true
- else if — checked only if above was false
- You can chain as many else if as needed
- else — the fallback if nothing matched
- Only ONE block ever runs

Outcome: Order matters — conditions are checked top to bottom, and the first true one wins.

The switch Statement

A cleaner alternative when checking one variable against many fixed values.

CODE

```
let day = "Mon";

switch (day) {
  case "Sat":
  case "Sun":
    console.log("Weekend");
    break;
  case "Mon":
    console.log("Bootcamp day!");
    break;
  default:
    console.log("Regular day");
}

// Output: Bootcamp day!
```

KEY RULES

- Never forget break — without it, execution 'falls through' into the next case
- Stacked cases (no break between) share one block
- default runs when nothing else matches
- Best for one variable, many exact values

Outcome: if/else if handles ranges and complex conditions; switch is cleaner for exact-value matching.

Loops — the for Loop

Repeats a block of code a known number of times.

CODE

```
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
}
// Count: 1 2 3 4 5
```

`let i = 1`	Initialization — runs once, sets the start
`i <= 5`	Condition — checked before every loop; stops when false
`i++`	Increment — runs after each loop iteration

REAL-WORLD USE

for loops power things you use daily: rendering a list of products, generating table rows, checking every item in a cart, or printing multiplication tables.

```
// Even numbers 1-10
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    console.log(i);
  }
}
```

Outcome: Use a for loop when you know how many times to repeat — like looping over a fixed list.

Loops — while & do-while

Repeats a block of code while a condition stays true.

WHILE

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

DO-WHILE

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);
```

WHILE vs DO-WHILE

- while checks the condition first — may run zero times
- do-while runs the block once first, then checks
- Use while when the count is unknown ahead of time
- Both need a variable that changes, or the loop never ends

Outcome: Forgetting to update the loop variable causes an infinite loop — always double-check it.

Real-World Example: Grade Calculator

Variables, operators, conditionals and a loop — all working together.

CODE

```
const marksList = [85, 42, 67, 90, 55];

for (let i = 0; i < marksList.length; i++) {
  let marks = marksList[i];
  let grade;

  if (marks >= 75) {
    grade = "A";
  } else if (marks >= 50) {
    grade = "B";
  } else {
    grade = "Fail";
  }

  console.log(`Student ${i + 1}: ${grade}`);
}
```

WHAT'S HAPPENING

- const holds a fixed list of marks
- for loop walks through each mark
- let grade is reassigned each round
- if / else if / else decides the grade
- Template string builds the output line

Best Practices & Common Mistakes

DO

- ✓ Default to const, use let only when needed
- ✓ Always use === and !== for comparisons
- ✓ Give variables clear, descriptive names
- ✓ Add a break in every switch case

DON'T

- ✗ Using var in new code
- ✗ Comparing with == instead of ===
- ✗ Forgetting to update the loop counter
- ✗ Writing deeply nested if/else instead of clean logic

Outcome: Clean variable use and strict comparisons prevent most beginner JavaScript bugs.

Hands-on Practice Task

Use part of the discussion window to build this along with the class.

BUILD: An Even/Odd & Grade Checker

1. Declare a const array of 5 numbers of your choice
2. Loop through the array using a for loop
3. Inside the loop, use % to check if each number is even or odd
4. Declare a separate let marks variable and grade it with if/else if/else
5. Print every result to the console with console.log()

DURING THE 30-MINUTE DISCUSSION WINDOW

Share your screen or drop your code in the Discord channel. We'll debug together, answer doubts, and review a few submissions live before wrapping up.

Today We Learned

JS

Introduction to JavaScript

role in the web, adding JS to a page

x

Variables & Constants

let, const, var, naming rules

#

Data Types

string, number, boolean, null, undefined

+

Operators

arithmetic, comparison, logical

?

Conditional Statements

if, else if, else, switch

↻

Loops

for, while, do-while

Outcome: You can now write basic JavaScript programs and implement simple decision-making logic.

Coming Up Next: Session 7

Next Saturday, same time — we build on today's logic foundation.

1 **Functions**

reusable blocks of logic, parameters & return values

2 **Arrays in Depth**

push, pop, map, filter and iteration methods

3 **Objects**

storing structured, real-world data

4 **DOM Introduction**

selecting and reading elements from HTML

5 **Events**

responding to clicks, input and page actions

6 **Mini Project**

an interactive to-do list using everything so far

Outcome: With functions, arrays and the DOM, you'll start making pages truly interactive.

Thank You!

Questions? Drop them in the Discord chat — happy to help.

PRACTICE BEFORE SESSION 7

Write a JavaScript program that stores 5 numbers in an array, loops through them, and prints whether each is even/odd and whether it's above or below 50 using if/else.

Session 7 – next Saturday, 4:00 PM, on Discord · we build on this with functions & arrays